SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Engineer to Excel

CO1-ASSESSMENT TOOL -1

NAME : APARNA T

REG NO :192524213

COURSE CODE : CSA0502

COURSE NAME : DATA STRUCTURES

1. Define Abstract Data Type (ADT) and Explain Its Features (5 Marks)

Definition

An Abstract Data Type (ADT) is a logical description of a data type that defines the data and the operations that can be performed on it, without specifying how those operations are implemented.

Example: Stack ADT supports operations such as push(), pop(), and peek().

Features of ADT

1. Abstraction: Hides the implementation details and shows only the required operations.
2. Encapsulation: Combines data and the operations performed on that data into a single logical unit.
3. Data Hiding: Prevents users from directly accessing internal implementation details.
4. Reusability: The same ADT can be reused in different programs.
5. Implementation Independence: An ADT can be implemented using different data structures without changing its basic operations.

Examples of ADT

- Stack
- Queue
- List
- Tree
- Array

Conclusion: ADTs simplify program development by separating the logical definition of data from its implementation.

2. Design an ADT for Array and Implement It in C with Complexity Analysis (10 Marks)

1. Definition

An Array ADT is a collection of elements of the same data type stored in consecutive memory locations. Each element is accessed using an index.

2. Operations of Array ADT

The main operations are:

1. Create: Creates an array.
2. Insert: Adds an element at a specified position.
3. Delete: Removes an element from a specified position.
4. Search: Finds an element in the array.
5. Display: Prints all the elements.

3. C Program

The following program implements the Array ADT using insertion, deletion, searching, and display operations.

```
#include <stdio.h>


int main()
{
   int a[100], n, i, pos, value, key;


   printf("Enter array size: ");
   scanf("%d", &n);
```

```
printf("Enter elements: ");
for(i = 0; i < n; i++)
    scanf("%d", &a[i]);

// Display
printf("Array: ");
for(i = 0; i < n; i++)
    printf("%d ", a[i]);

// Insert
printf("\nEnter position and value to insert: ");
scanf("%d %d", &pos, &value);

if(pos >= 1 && pos <= n + 1 && n < 100)
{
    for(i = n; i >= pos; i--)
        a[i] = a[i - 1];

    a[pos - 1] = value;
    n++;
}
else
    printf("Invalid position or array full\n");

printf("After insertion: ");
for(i = 0; i < n; i++)
    printf("%d ", a[i]);

// Delete
printf("\nEnter position to delete: ");
```

```
scanf("%d", &pos);

if(pos >= 1 && pos <= n)
{
   for(i = pos - 1; i < n - 1; i++)
      a[i] = a[i + 1];

   n--;
}
else
   printf("Invalid position\n");

printf("After deletion: ");
for(i = 0; i < n; i++)
   printf("%d ", a[i]);

// Search
printf("\nEnter element to search: ");
scanf("%d", &key);

for(i = 0; i < n; i++)
{
   if(a[i] == key)
   {
      printf("Element found at position %d", i + 1);
      break;
   }
}

if(i == n)
```

```
        printf("Element not found");


    return 0;
}
```

4. Sample Input

Enter array size: 4

Enter elements: 10 20 30 40

Enter position and value to insert: 2 15

Enter position to delete: 3

Enter element to search: 30

5. Sample Output

Array: 10 20 30 40

After insertion: 10 15 20 30 40

After deletion: 10 15 30 40

Element found at position 3

6. Complexity Analysis

Let n be the number of elements in the array.

Operation	Time Complexity	Explanation
Access	O(1)	Accesses an element directly using its index.
Display	O(n)	Visits every element.
Insertion	O(n)	May require shifting elements.
Deletion	O(n)	May require shifting elements.
Searching	O(n)	May examine every element.

Space Complexity: O(n), because the array stores up to n elements.

7. Advantages of Array ADT

- Provides fast access using an index.
- Stores elements of the same data type.
- Is easy to traverse and process.
- Uses contiguous memory locations.

Conclusion: Array ADT provides operations to store, insert, delete, search, and access elements efficiently.

3. Explain the Role of Performance Analysis in Real-Time Systems (5 Marks)

Definition

Performance analysis is the process of evaluating a system's execution time, memory usage, and resource utilization to ensure that it meets its performance requirements.

A real-time system must produce the required result within a specified time limit.

Examples: Airbag control systems, medical monitoring systems, and traffic signal control systems.

Role of Performance Analysis

1. Execution Time: Measures how long a task takes to execute and checks whether it finishes within the deadline.
2. Response Time: Determines how quickly the system responds to an event or input.
3. Resource Utilization: Analyses the use of CPU, memory, and other system resources.
4. Deadline Management: Ensures that important tasks are completed within their specified deadlines.
5. Reliability and Safety: Helps identify delays and performance problems that could cause system failure.

Example

In a car airbag system, the airbag must activate within a very short time after a collision is detected. Performance analysis helps verify that the system responds within the required deadline.

Conclusion

Performance analysis is essential in real-time systems because it ensures timely execution, efficient resource utilization, reliability, and safety.